

FoodieGo

Technical Specification & Deployment Manual

Cloud Computing Capstone Project

Level 4 — IS Department

Student ID	Student Name
2022170043	أحمد مصطفى عبدالراضي حسين
2022170595	كمال الدين سيد كمال عودة
2022170283	عمر محمد سيد احمد
2022170265	عمر احمد عباس عبدالعليم
2022170457	ميناء توفيق حماية
2022170590	عمر محمد وفيق ذكي محمد العطار

Academic Year 2025/2026

Contents

1 Abstract	2
2 Architectural Design Rationale	3
2.1 Microservices Decomposition	3
2.2 Communication Protocols	3
2.3 Data Consistency: Database-per-Service	3
3 Infrastructure & Containerization Strategy	4
3.1 Development Workflow	4
3.2 Optimized Image Engineering	4
4 Advanced Kubernetes Platform Engineering	5
4.1 Persistence & State Management	5
4.2 High Availability & Self-Healing	5
4.3 Distributed Log Aggregation (DaemonSets)	5
4.4 Network Topology	5
5 Cloud Observability	6
5.1 Metrics & Centralized Logs	6
5.2 Correlation & Troubleshooting	6
6 System Reliability & Security	7
6.1 Infrastructure Security: RBAC	7
6.2 Application Security	7
7 Deployment & Operational Manual	8
7.1 Kubernetes Manifest Inventory	8
7.2 Deployment Instructions	8
8 Validation & Testing	9
9 Conclusion	10

Chapter 1

Abstract

This project report details the development and deployment of **FoodieGo**, a cloud-native food delivery system designed with a microservices-oriented architecture. The system's primary objectives are to demonstrate high scalability, robust fault isolation, and full-stack observability within a containerized environment. By decoupling the platform into four discrete functional services—User, Restaurant, Order, and Payment—the architecture facilitates independent scaling and deployment. The system leverages Docker for containerization and Kubernetes for orchestration, utilizing an advanced observability stack comprising Prometheus, Grafana, and Loki. The result is a highly resilient platform capable of maintaining operational transparency through centralized logging and real-time metrics.

Chapter 2

Architectural Design Rationale

2.1 Microservices Decomposition

The system architecture follows a functional decomposition strategy, splitting the platform into four independent microservices. This separation is justified by the distinct operational boundaries and scaling requirements of each domain:

- **User Service:** Manages identity, authentication (JWT), and driver metadata.
- **Restaurant Service:** Handles catalog management and menu lifecycle.
- **Order Service:** Acts as the business orchestrator, coordinating cross-service validations.
- **Payment Service:** Processes transactions asynchronously to ensure non-blocking order flows.

2.2 Communication Protocols

The platform employs a hybrid communication strategy to optimize for both latency and reliability. Synchronous RESTful APIs (via Axios) are utilized for user-facing flows where immediate response is required, such as fetching menus or validating user credentials. Conversely, the Advanced Message Queuing Protocol (AMQP) via RabbitMQ is implemented for the payment pipeline. This asynchronous approach decouples the Order Service from the Payment Service, allowing the system to handle bursts in order volume without overwhelming the payment processor.

2.3 Data Consistency: Database-per-Service

To prevent "tight coupling" at the data layer, the system implements the ****Database-per-Service**** pattern. While hosted on a single PostgreSQL 16 instance for resource efficiency, strict isolation is enforced through four distinct schemas: `user_svc`, `restaurant_svc`, `order_svc`, and `payment_svc`. This prevents direct cross-service table joins and ensures that data is only accessible via authorized service APIs, maintaining high modularity.

Chapter 3

Infrastructure & Containerization Strategy

3.1 Development Workflow

The project utilizes **Docker Compose** as the primary vehicle for reproducible local environments. Three distinct configurations are maintained—Dev, Test, and Prod—ensuring that the development lifecycle remains consistent across different environments. The use of volume mounts in development allows for “hot-reloading” (via *nodemon*), enabling rapid iteration without constant rebuilding of images.

3.2 Optimized Image Engineering

To adhere to cloud-native security best practices, all services employ **multi-stage Docker builds**. This technique separates the build-time environment (where dependencies are fetched and assets compiled) from the runtime environment. By copying only the necessary production binaries and `node_modules` into a minimal Alpine-based runtime image, the system reduces the attack surface and minimizes image size, leading to faster pulls and reduced storage costs.

Chapter 4

Advanced Kubernetes Platform Engineering

4.1 Persistence & State Management

Stateful components such as PostgreSQL, RabbitMQ, and the observability databases (Grafana/Loki) are deployed using **StatefulSets**. Unlike standard Deployments, StatefulSets provide stable network identifiers and persistent storage bindings. These are backed by **PersistentVolumeClaims (PVCs)**, ensuring that critical data survives pod rescheduling or cluster node failures.

4.2 High Availability & Self-Healing

The system achieves high availability through the configuration of **Readiness** and **Liveness probes**. Readiness probes ensure that the load balancer (Service) only routes traffic to pods that have successfully initialized (e.g., connected to the database). Liveness probes allow the Kubernetes Kubelet to detect deadlocked or unhealthy pods and automatically restart them, providing an autonomous self-healing mechanism.

4.3 Distributed Log Aggregation (DaemonSets)

Logging in a distributed system is challenging as logs are ephemeral. FoodieGo implements **Promtail** as a **DaemonSet** to solve this. Running one Promtail pod per cluster node ensures that logs from every container on the node are automatically scraped from the host filesystem (`/var/log/pods`) and shipped to a centralized Loki server.

4.4 Network Topology

The cluster network is designed to separate internal and external concerns. Most backend services use **ClusterIP**, making them reachable only within the cluster. Administrative dashboards and the Frontend application are exposed via **NodePort** services (e.g., NodePort 30080 for Frontend, 30001 for Grafana), providing controlled external access.

Chapter 5

Cloud Observability

5.1 Metrics & Centralized Logs

The observability stack is built on the **PLG (Promtail, Loki, Grafana)** and Prometheus framework. **Prometheus** performs automated service discovery by querying the Kubernetes API for pods annotated with `prometheus.io/scrape`. It collects resource metrics (CPU/Memory) via **cAdvisor** and application-level metrics via the Node.js `prom-client` library.

5.2 Correlation & Troubleshooting

By integrating Loki (logs) and Prometheus (metrics) into pre-provisioned **Grafana Dashboards**, the system allows engineers to correlate performance spikes with application errors. For example, a sudden increase in CPU usage on the Order Service can be immediately investigated by pivoting to the associated Loki logs using **LogQL** in the same interface, significantly reducing the Mean Time to Recovery (MTTR).

Chapter 6

System Reliability & Security

6.1 Infrastructure Security: RBAC

To protect the cluster API, the system implements **Role-Based Access Control (RBAC)**. Infrastructure agents like Prometheus and Promtail are assigned specific `ServiceAccounts` linked to `ClusterRoles` that grant only the "Least Privilege" necessary—such as the ability to "list" and "watch" pods without having access to sensitive secrets or the ability to modify workloads.

6.2 Application Security

At the application layer, the system utilizes **JSON Web Tokens (JWT)** for stateless authentication. Passwords are never stored in plain text; the **Bcrypt** hashing algorithm with a salt factor of 10 is used to ensure credential security. Furthermore, the Order Service performs a "Price Integrity Check" against the Restaurant Service for every order to prevent client-side price tampering.

Chapter 7

Deployment & Operational Manual

7.1 Kubernetes Manifest Inventory

The system is defined by 26 declarative manifests.

Table 7.1: Kubernetes Manifest Catalog

Category	Manifests	Description
Config/Secrets	00, 01	Environment variables, DB creds, and JWT secrets.
StatefulSets	02, 14, 19, 24	Persistent workloads for DB, RabbitMQ, Grafana, and Loki.
Deployments	04, 06, 08, 10, 12, 17, 22	Stateless microservices and monitoring engines.
DaemonSets	25	Promtail log collection agent.
Services	03, 05, 07, 09, 11, 13, 15, 18, 20, 24	Networking endpoints (ClusterIP and NodePort).
RBAC	21, 25	Security permissions for scraping and logging.

7.2 Deployment Instructions

Deployment to a cluster (e.g., Minikube) follows these steps:

1. Build images: `docker build -t food-delivery/user-service:latest ./services/user-service` (repeat for all).
2. Apply manifests: `kubectl apply -f k8s/`
3. Verify readiness: `kubectl get pods --watch`
4. Access application: `minikube service frontend-service`

Chapter 8

Validation & Testing

System integrity is validated using an **End-to-End (E2E) Test Suite** containing 44 assertions. This suite simulates the entire customer journey:

- **Authentication:** Registration and login flows.
- **Ordering:** Cart management, price validation, and RabbitMQ message production.
- **Asynchronicity:** Verification of transaction records in the Payment Service post-order.
- **Negative Testing:** Ensuring invalid JWTs and unauthorized role access are rejected.

Chapter 9

Conclusion

The FoodieGo project successfully demonstrates the efficacy of microservices and Kubernetes in building modern, scalable applications. By prioritizing observability and declarative infrastructure, the platform remains maintainable even as complexity increases. The transition from monolithic patterns to decoupled, event-driven services, combined with automated logging and monitoring, establishes a robust foundation for enterprise-level cloud deployments.